Programmation GPGPU avec CUDA - TP Projet

Ligne de vue

maxime.maria@unilim.fr
cyprien.plateauholleville@unilim.fr
vincent.larroque@unilim.fr

Les cartes de hauteur sont couramment utilisées dans les systèmes d'information géographiques notamment, car elles sont plus simples à obtenir et à manipuler qu'une scène 3D tout en proposant un niveau d'information comparable. Ces images ne proposent en général qu'un seul canal contenant la hauteur associée au pixel actuel comme illustré sur la figure 1.

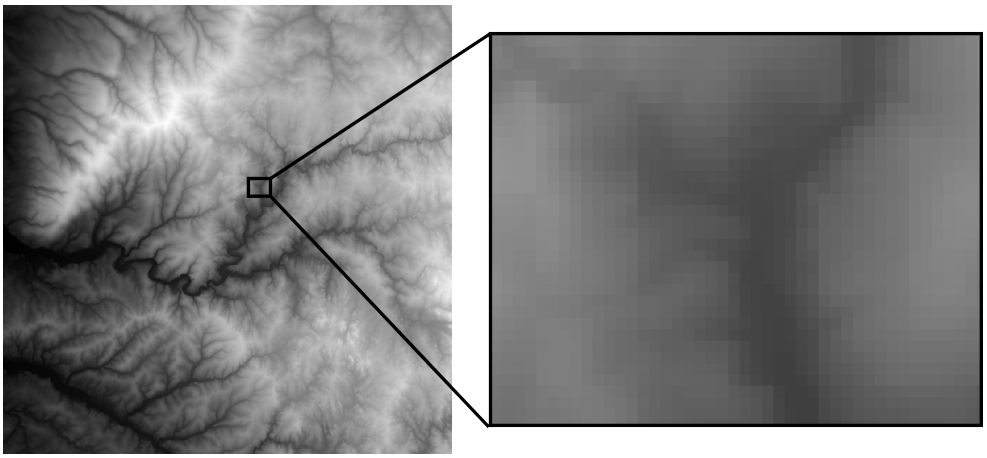


FIGURE 1 – Figure représentant une carte de hauteur dans le limousin. Source : https://geoservices.ign.fr/bdalti, rendu effectué avec : https://www.qgis.org/fr/site/

Il est parfois intéressant de connaitre quels sont les points $p_{i,j}$ visibles à partir d'un point c choisi dans l'image. Cette information peut être obtenue en testant si un pic est localisé entre les deux. De façon plus précise, si $\mathcal{L}_{i,j} = \{p_0,\ p_1,\ \ldots,\ p_{n-1}\}$ est l'ensemble des points entre c et $p_{i,j}$, $p_{i,j}$ est visible depuis c si et seulement si :

$$\measuredangle(c,p) < \measuredangle(c,p_{i,j}),\ \forall p \in \mathcal{L}_{i,j}$$

avec $\measuredangle(a,b)$ la fonction donnant l'angle entre le segment $\vec{ab}$ et le plan $\mathcal{P}_{xy}$, pour $a = \begin{pmatrix} x_a \\ y_a \\ z_a \end{pmatrix}$ et $b = \begin{pmatrix} x_b \\ y_b \\ z_b \end{pmatrix}$, définie par :

$$\measuredangle(a,b) = \arctan\left(\frac{z_b - z_a}{\sqrt{(x_b - x_a)^2 + (y_b - y_a)^2}}\right)$$

Pour obtenir les cases entre c et $p_{i,j}$, on effectue une rastérisation de la droite $p_{i,j}c$ sur l'image avec un algorithme de type analyseur différentiel numérique (DDA : `https://fr.wikipedia.org/wiki/Analyseur_diffrentiel_numrique`). Soit deux points $p_1 = (x_1, y_1)$ et $p_2 = (x_2, y_2)$, DDA donne les D coordonnées par lesquelles la droite passe, avec $D = \max(|x_2 - x_1|, |y_2 - y_1|)$. La ième case entre $p_{i,j}$ et c peut ainsi être obtenue à partir de :

$$\begin{pmatrix} x_i \\ y_i \end{pmatrix} = \begin{pmatrix} x \\ y \end{pmatrix} + \begin{pmatrix} step_x \\ step_y \end{pmatrix} i$$

avec $step_x = (x_2 - x_1)/D$ et $step_y = (y_2 - y_1)/D$.

Puisqu'un tel traitement peut rapidement devenir coûteux, notamment sur les cartes de hauteur de grandes tailles, il est intéressant de le paralléliser sur GPU.

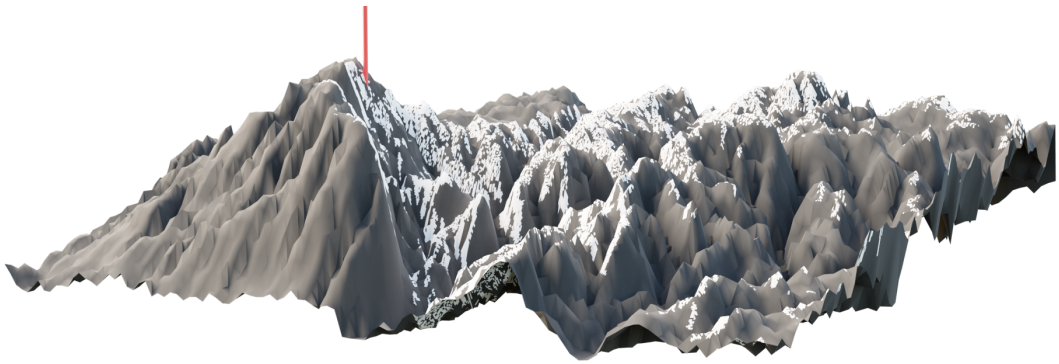


FIGURE 2 – Image 3D de l'image `1.input.ppm` colorée à partir de `2.result.ppm`. La partie blanche représente ce qui est visible depuis le point $(245, 497)$.

Consignes

L'objectif de ce projet est de programmer en CUDA un algorithme qui donnera, à partir d'un point et une image donnés, tous les points visibles à partir de celui-ci. **Nous attendons que vous appliquiez les notions d'optimisation vues durant les séances précédentes.**

S'il y a des optimisations à faire, particulièrement en CUDA, elles doivent avoir du sens et être justifiées un minimum. **Important :** quatre algorithmes doivent être codés obligatoirement, mais vous avez le droit de laisser les différentes versions écrites dans le code final. Cela permet de voir l'évolution du processus d'optimisation ou de compréhension du problème. Vous pouvez ajouter des suffixes aux noms des fonctions pour indiquer tout cela.

Ce travail est à réaliser en binôme.

À coder

Vous disposez de la classe `Heightmap` pour pouvoir charger et sauvegarder des images au format `.ppm` ainsi que deux cartes de hauteur `limousin-tile.ppm` et `limousin-full.ppm`. La première est de petite taille et sera préférable pour les premiers tests tandis que la seconde donnera une meilleure indication des temps de calcul.

1. Implémentez un kernel `naive_viewtest` qui, pour chacun des pixels de l'image, calcule sa visibilité depuis le centre c donné en paramètre.
2. Le premier kernel est peu efficace, il répète en effet de nombreux calculs. Proposez un kernel `optimized_viewtest` qui optimise les traitements en utilisant les notions vues en cours.

Même si nous avons optimisé les traitements du calcul précédent, il peut être intéressant de travailler sur une image plus simple que celle qui nous est donnée. Cela peut être effectué en créant un image de résolution plus petite à partir d'une découpe de l'image de départ et en combinant la contribution de plusieurs pixels.

1. Implémentez un kernel `naive_tiled` qui construit l'image de plus petite résolution à partir des hauteurs maximales de l'image dans la tuile correspondante. Vous utiliserez une opération atomique pour chacun des pixels.
2. Le premier kernel est peu efficace, proposez une version améliorée `optimized_tiled` en utilisant les concepts vus en cours.

N.B. Il serait judicieux de commencer par la version séquentielle. Nous vous proposons enfin des résultats de référence pour chacun des algorithmes à implémenter, n'hésitez pas à comparer avec vos résultats. `2.result.ppm` est le résultat attendu de l'implémentation de l'exercice 1 sur l'image `1.input.ppm` et avec $c = (245, 497)$ tandis que `3.tiled.ppm` est celui attendu pour l'exercice 2 en utilisant une découpe en 10x10. La figure 2 illustre ces résultats.

À rendre

Vous devez rendre un dossier nommé `GPGPU_Nom1_Nom2.zip` contenant :

1. **10 points** - Votre code qui doit compiler et fonctionner sur les machines de la fac (I211/I212) ;
2. **10 points** - Un rapport au format PDF dans lequel vous expliquerez votre code et **justifierez vos choix**. Vous discuterez notamment de la répartition des *threads* sur la grille de calcul. Vous comparerez les versions séquentielles et parallèles. Si vous avez plusieurs versions d'un même *kernel*, comparez les performances et analysez-les (*e.g.* utilisation de la mémoire partagée ou non, différentes méthodes de synchronisation...).

Attention : Comme noté ci-dessus, la répartition des points entre le rapport et le code est homogène. Une très bonne implémentation sans analyse pertinente (ou l'inverse) ne permettra donc pas d'obtenir une bonne note. Il vous est fortement conseillé d'utiliser les méthodes d'analyse du TP3-Histogramme (Génération automatique des benchmarks, production de courbes permettant une analyse plus facile, etc.).

Vous devez rendre votre travail avant le **22 décembre 2023, 23h59**.
Tout retard entraînera une pénalité de $2^{nombre_de_jours_de_retard}$ points.

Attention au plagiat !

Vous le savez, vous avez même signé un engagement concernant le plagiat/la fraude à l'Université... Mais une piqûre de rappel ne fait pas de mal ! Faites très attention, la fraude peut entraîner jusqu'à l'exclusion du système universitaire français !